

Machine Learning in Computational Mechanics

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

9. Juli 2024

1 General explanation of the source code

The given source code implements a Physics-Informed Neural Network (PINN) solving a two-dimensional elastic problem. The model utilizes both neural network capabilities and physical laws to predict the deformation of a rectangle. The left boundary of the rectangle is fixed while the right boundary has a load applied, which is a so called Neumann boundary condition having specified a force per area (traction). This case is also referred as a tensile test or uniaxial tension test.

After the setup of the utilized libraries and boundary conditions, a meshgrid with sample points is generated and the neural networks architecture is defined. Furthermore a custom activation function squaring its input and adding a linear component depending on the trainable parameter slope is defined. This enables non-linear transformations within the network.

The strain tensor for every sample point is calculated by computing the gradients of the displacement field. Afterwards the stress tensors are computed using the material law. The code implements a linear elastic material model for isotropic materials, based on Hooke's Law. This law relates the stress tensor ($\underline{\underline{\sigma}}$) to the strain tensor ($\underline{\underline{\varepsilon}}$) using the material's elastic constants which were defined earlier. The Lam parameters μ and λ are derived from the Young's modulus E and the Poisson's ratio ν , defining the linear elastic material behaviour:

$$\mu = \frac{E}{2(1 + \nu)}$$
$$\lambda = \frac{E\nu}{(1 + \nu)(1 - 2\nu)}$$

For isotropic materials, the stress-strain relationship can be written as:

$$\underline{\underline{\sigma}} = 2\mu\underline{\underline{\varepsilon}} + \lambda\underline{\underline{\mathbf{I}}}\text{tr}(\underline{\underline{\varepsilon}})$$

$$\sigma_{ij} = 2\mu\varepsilon_{ij} + \lambda\delta_{ij}\varepsilon_{kk}$$

Where:

- σ_{ij} is the stress tensor.
- ε_{ij} is the strain tensor.
- μ is the shear modulus (also known as the second Lam parameter).
- λ is the first Lam parameter.
- δ_{ij} is the Kronecker delta (which is 1 when $i = j$ and 0 otherwise).
- $\text{tr}(\underline{\underline{\varepsilon}}) = \varepsilon_{kk}$ is the trace of the strain tensor, i.e., the sum of the diagonal elements of $\underline{\underline{\varepsilon}}$.

In the code, the Deep Energy Method is utilized to minimize the total potential energy and thus approximating a displacement field. Using automatic differentiation, the strain tensors $\underline{\underline{\varepsilon}}$ are computed from the displacement field U . Afterwards the stress tensors and the strain energy density for each sample point are set up based on the strain tensors and the material model. The integration of the strain energy density over the domain yields the total potential energy. By applying the finite differences method, the divergence of the stress tensor is computed and then results in the divergence loss by calculating the Mean Squared Error (MSE) loss between the computed divergence and zero. To check the energy equilibrium between internal (strain) energy and external (traction) energy, the traction energy is computed from the boundary forces. Within the calculation of the boundary losses, the displacement boundary condition on the left boundary

and the traction boundary condition are enforced to ensure their fulfillment. Yet again the boundary losses are calculated by using MSE.

The total loss then yields from adding the difference of external traction energy and internal strain energy, the boundary loss and the divergence loss. In the given code the divergence loss is not included in the total loss.

To enable transfer learning between a data-driven training and physics-informed training, the loss can also be computed as the MSE between the predicted displacement and analytically calculated target values. These target values are approximated by calculating the total one dimensional elongation using Hooke's law first. Secondly the displacements in x- and y-direction are computed accounting the poisson effect. The data-driven approach uses the MSE loss between the predicted displacement and the approximated target values to create good initial values of the training parameters and ensure initial convergence as well as higher initial accuracy. After the initial data-driven training phase the model transitions to physics-informed training to satisfy the physical laws incorporated in the loss function.

As an additional parameter the code provides the possibility to define a custom activation function containing a trainable parameter to adapt the activation function during the training process. In the training process the first 1000 of 10000 epochs are executed as data-driven training while the rest are transfer learning with physics-informed training. During the forward pass the predicted outputs of the displacements are computed. Afterwards the loss is calculated and forwarded to the backward pass, which computes the gradient of the loss with respect to the model parameters. The Adaptive Moment Estimation Optimizer (ADAM) performs a single step optimization using the computed gradient and outputs the optimized model parameters for the next iteration.

2 Analysis

In order to analyze the influences of the parts of the loss functions, there are some techniques for analyzing a loss function effectively, such as:

- plotting the Loss Curve
- hyperparameter tuning
- learning rate analysis

The model that will be analyzed is represented in Figure 2.1. The formulation of the loss can be classified into two different cases:

- physics informed loss corresponding to the potential energy of the continuum, which can be calculated as an integral extended over the computational domain,
- data-driven loss, which is computed considering the MSE between target values and predictions from the NN.

Note that both are scalar values, which is required to properly define the loss in the context of neural networks.

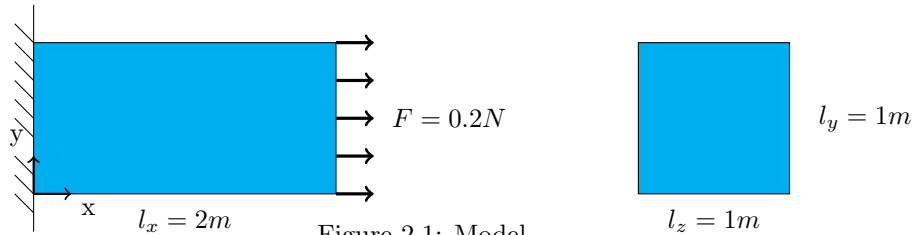


Figure 2.1: Model

In the code, an equivalent formulation of the boundary value problem is defined by $\nabla \cdot \sigma \stackrel{!}{=} 0$, which needs to be fulfilled, along the minimization of potential energy.

2.1 Influence of epochs

Firstly, we analyse the given Code with 1000 epochs up to 10000 epochs to see the behaviour of deformation. The analytic of deformation can be calculated with $F = 0.2N, l = 2m, E = 1Pa, A = 1m^2$:

$$U_x = \frac{F \cdot l}{E \cdot A} = 0.4m$$

In Figure 2.2, deformation using DEM is changing by increasing epochs. At low number of epochs, the predictions for deformation might be highly inaccurate as the weights in neural network are still far from optimal. By increasing epochs, the results begin to better approximate the expected solution, thanks to the backpropagation process, which is aimed to actually minimize the potential energy. For the next analysis in this section, the model will be trained for 1000 epochs, after which the training of the model results in more marginal improvements, see Figure 2.2b.

2.2 Influence of boundary loss

To calculate the prediction of the deformation field, the boundary loss is important in the DEM, which is designed to enforce the boundary conditions of the physical problem being modeled, see Figure 2.1. Without boundary loss, the model might predict solutions that violate boundary conditions, which lead to incorrect deformation fields. The process of balancing the boundary loss with other loss components must be tuned carefully to ensure that the boundary conditions are enforced without the energy minimization objective taking over the training process. Therefore, Figure 2.3 show the results by activating just `loss_right` as boundary loss. To get a better understanding, the results will be compared with the enforcement of loss components (`loss_right`, `loss_upper` and `loss_lower`), which is shown in Figure 2.5. In Figure 2.3b, 2.3c, 2.3d, the normal and transversal traction have higher value at early epochs, while in Figure 2.4c, 2.4d, 2.4e around 200 epochs has highest value. The deformation U_x of DEM in Figure 2.4a is not yielding good results. In Abaqus, the strain energy ALLSE is $\Psi = 0.040$, which is smaller than analytic value in the code $\Psi = 0.09315$, see Figure 2.3e.

2.3 Influence of divergence loss

Figure 2.5 illustrates the results of traction with the inclusion of divergence losses. The divergence loss component is critical as it minimizes the divergence of the stress tensor, leading to more accurate and physically consistent predictions. In the displacement plot, the inclusion of divergence losses results in the predicted U_x deviating less from the analytic solution compared to the scenario without divergence losses. This inclusion helps maintain a more accurate displacement field, stabilizing the training process and ensuring that the model's predictions adhere closely to the expected analytic solution. The influence of divergence loss on the loss components is significant. The residual loss no longer mirrors the boundary loss, indicating that the additional constraint imposed by the divergence loss affects the overall residual loss. Without this constraint, the residual loss primarily reflects the boundary loss. The T-PSI loss values are higher when divergence losses are considered. The curve appears to be more stable, exhibiting fewer spikes. The boundary loss shows similar behavior in both scenarios but is more stable with the inclusion of divergence loss. The divergence loss decreases significantly when is activated as a loss term, indicating the model's learning to satisfy the divergence-free condition. This contributes to a more robust training process by preventing overfitting to boundary conditions and encouraging a globally consistent solution. The traction plots reveal different dynamics for the transversal and normal components with divergence loss,

contrasting with the parallel trajectories observed without it. This indicates a more complex and accurate representation of the traction components. The divergence loss also stabilizes the convergence behavior of traction components, achieving a more physically accurate model.

2.4 Influence of data driven approach

From now on, the behaviour NN architecture with data driven/physics informed combined training is considered. Figure 2.6a shows a better accurate deformation between analytic and DEM without influence of divergence Loss. This approach improves accuracy, generalization, and convergence while ensuring that the model remains robust and reliable, see Figure 2.6b.

2.5 Influence of the traction energy calculation method

The traction energy at the right boundary can either be calculated by multiplying the displacement by the defined force or by multiplying the displacement by the stress tensor. Using the force assumes a uniform force boundary while using the stress is more accurate especially with non-linear stress distributions.

To analyze the impact of the two different approaches, the predicted traction T_{NN} is compared with the analytically approximated traction T_{an} in Figure 2.7. To provide a analytical traction, the initial force at the right boundary is multiplied with the analytically approximated displacement at the right boundary.

To achieve a satisfying convergence of the traction components, the difference of analytical and predicted traction should aim towards zero. As displayed in Figure 2.7a calculating the traction by multiplying the displacement with the initial force neither leads to convergence nor great similarity of analytical and predicted traction. In contrast to that Figure 2.7b shows very similar results for analytical and predicted traction indicated by the graphs tending to zero for both normal and transversal component. According to the presented results the traction is calculated using the stress for the following investigations.

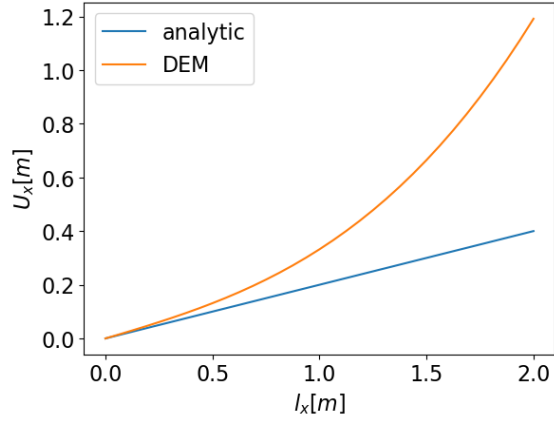
2.6 Influence of the incremental application of the force boundary condition

The code provided includes the possibility to apply the force incrementally. To investigate the difference between the two approaches, the force is increased incrementally and reaches its full extend at around 1200 epochs as shown in Figure 2.8b. Accordingly, the area below 1200 epochs is not meaningful, because the force is not fully applied here.

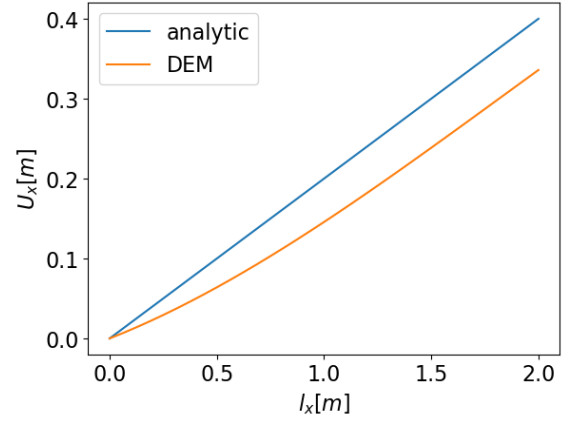
As displayed in Figure 2.8a the predicted strain energy does not converge to the value determined by FEM when applying the force in only one step. In contrast to that the incremental application of the force according to Figure 2.8b leads to really good convergence as the predicted value of the strain energy (orange) matches almost perfectly with the FEM result (green).

In general the incremental application improves the numerical stability by preventing large, abrupt changes in the model predictions. For small increments in force, the strain and displacement are also small which is consistent with the linear assumptions of the small deformation theory made in this case. This makes it easier for the model to learn the correct relationship between force and displacement, leading to results that converge with the FEM, which assumes small deformations for linear formulations as well.

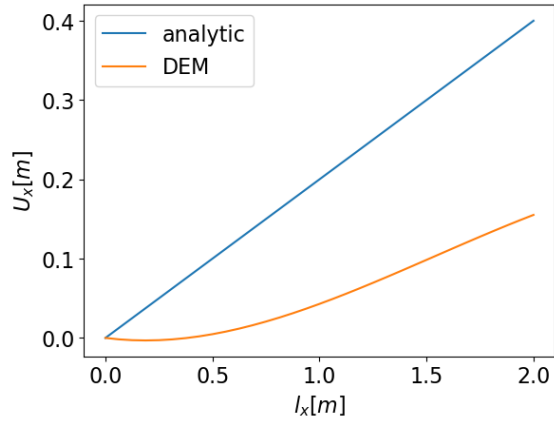
2.7 Results



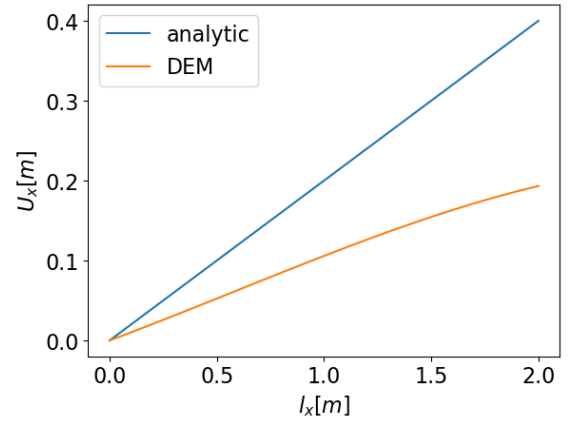
(a) Deformation U_x



(b) Deformation U_x by 1000 epochs

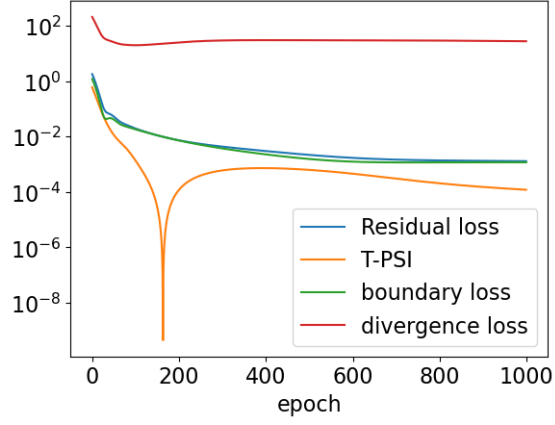


(c) Deformation U_x by 5000 epochs

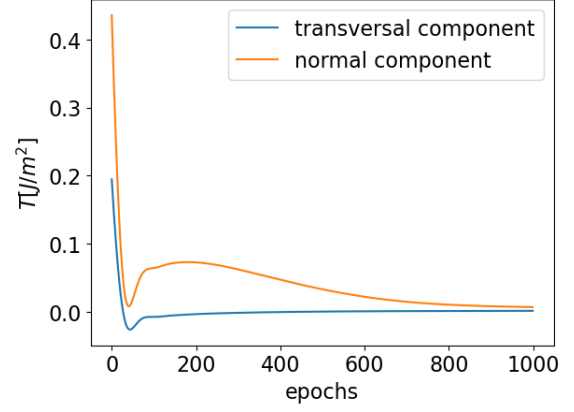


(d) Deformation U_x by 10000 epochs

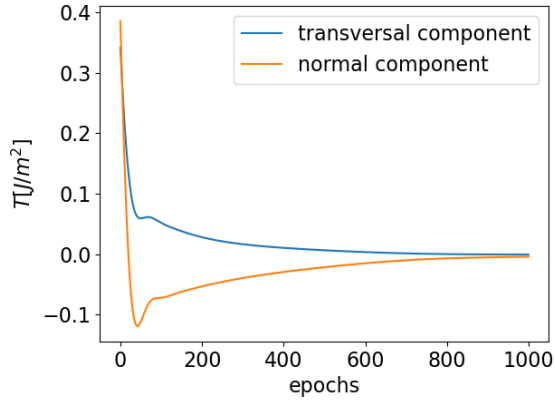
Figure 2.2: Analytic vs DEM deformation U_x



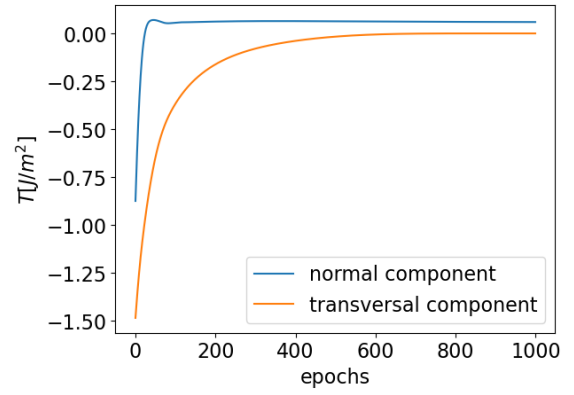
(a) Residual-, T-PSI, boundary, divergence Loss



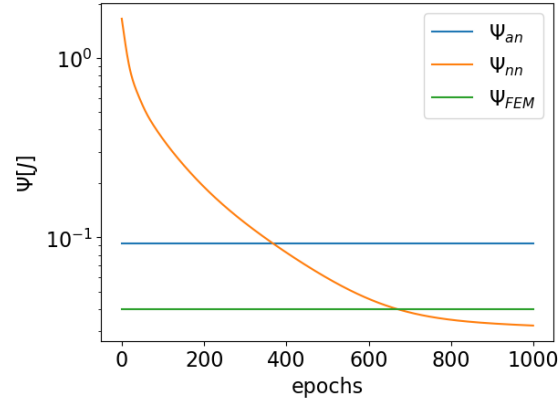
(b) Normal and Transversal Traction on lower face



(c) Normal and Transversal Traction on upper face

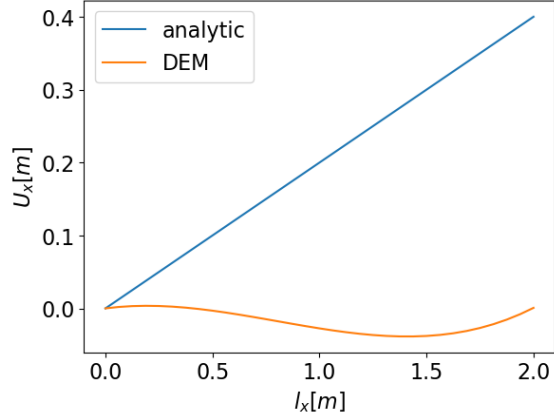


(d) Normal and Transversal Traction on right face

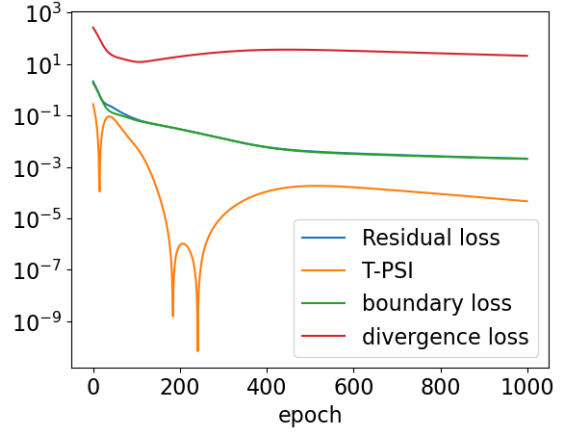


(e) strain energy Ψ

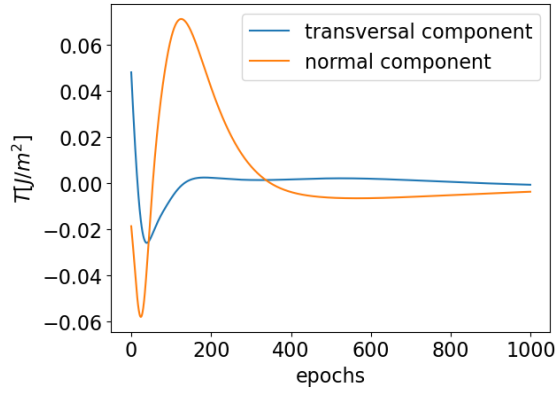
Figure 2.3: Loss, Total energy, Traction with right loss observation



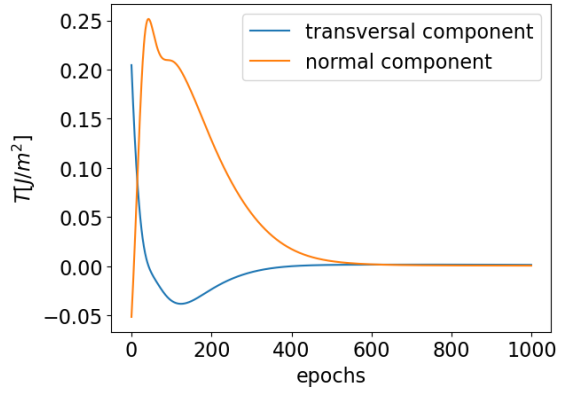
(a) Deformation U_x by 1000 epochs



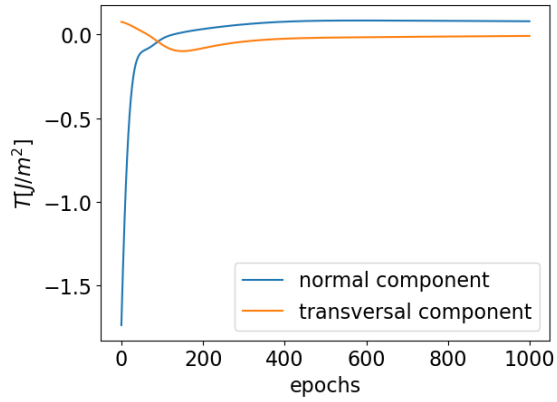
(b) Residual-, T-PSI, boundary, divergence Loss



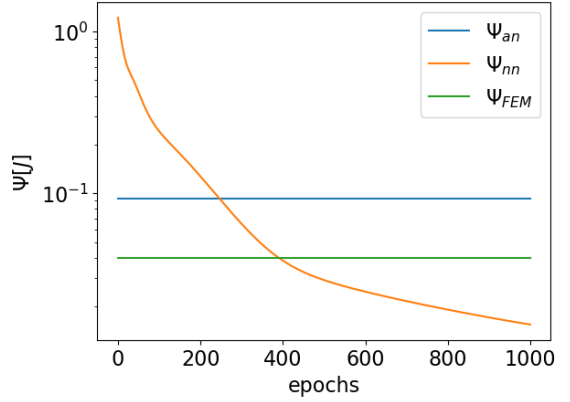
(c) Normal and Transversal Traction on lower face



(d) Normal and Transversal Traction on upper face

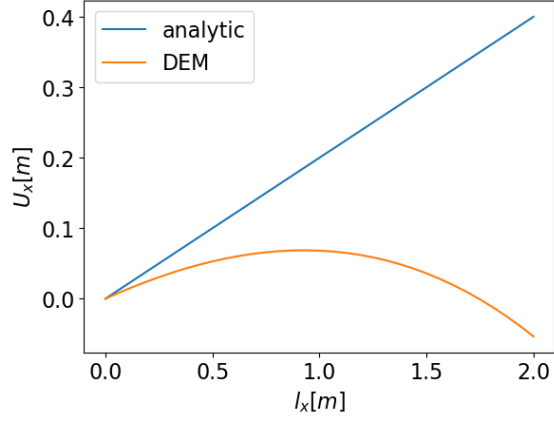


(e) Normal and Transversal Traction on right face

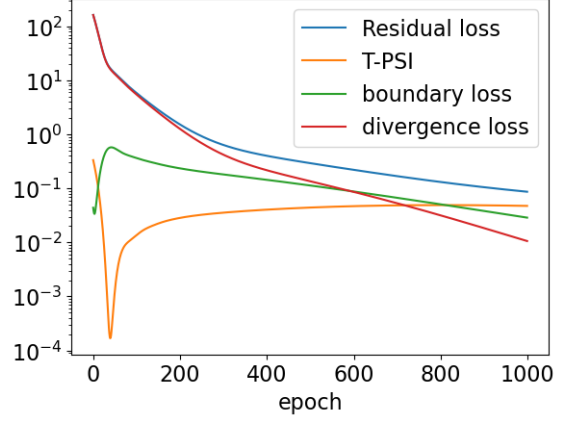


(f) strain energy Ψ

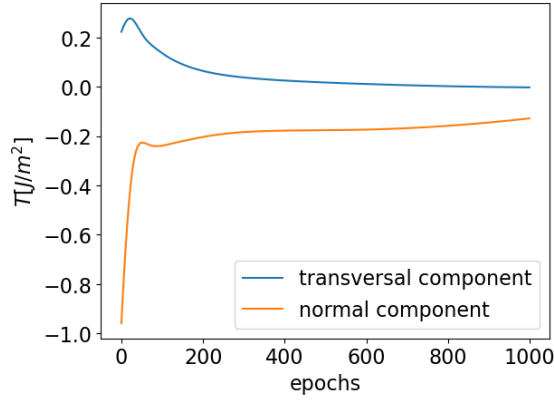
Figure 2.4: Loss, Total energy, Traction with all size loss



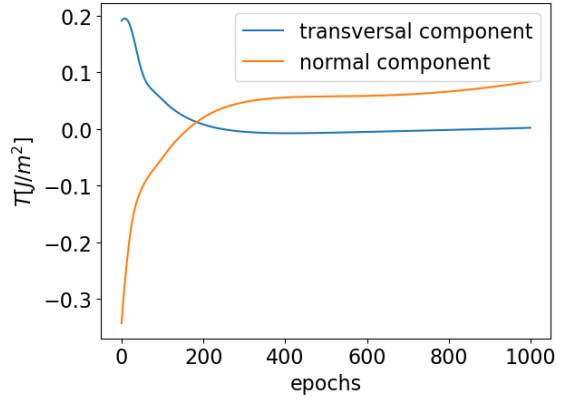
(a) Deformation U_x by 1000 epochs



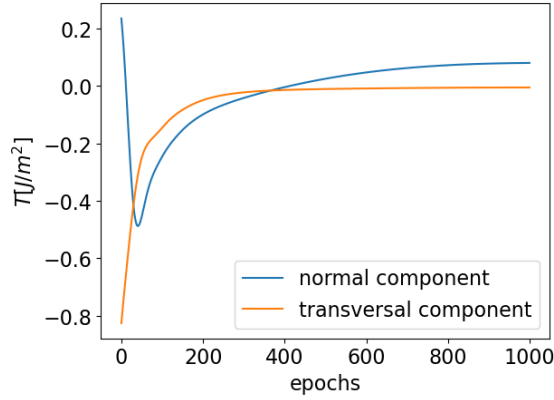
(b) Residual-, T-PSI, boundary, divergence Loss



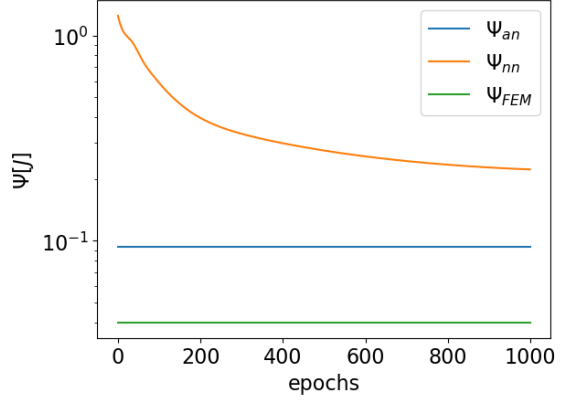
(c) Normal and Transversal Traction on lower face



(d) Normal and Transversal Traction on upper face

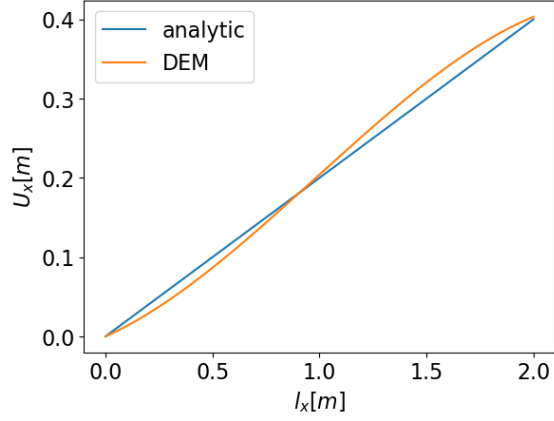


(e) Normal and Transversal Traction on right face

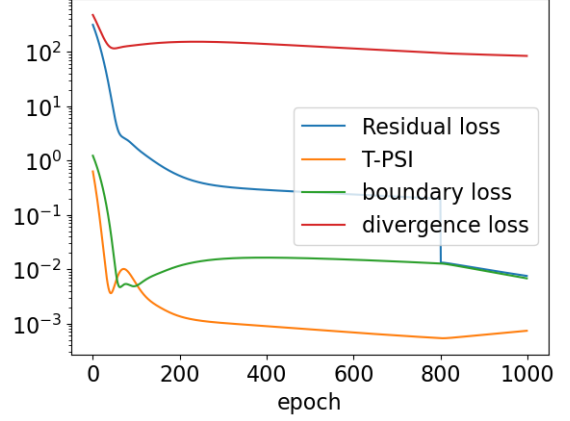


(f) strain energy Ψ

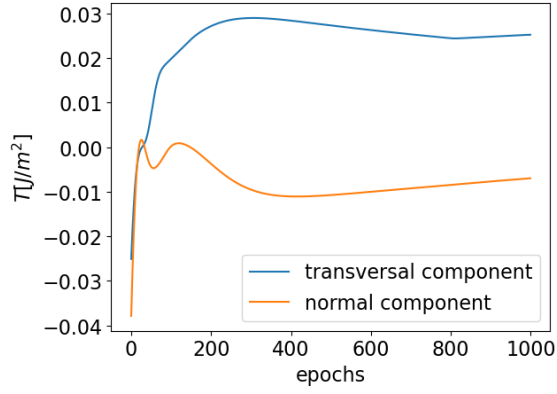
Figure 2.5: Loss, Total energy, Traction with influence div. Loss



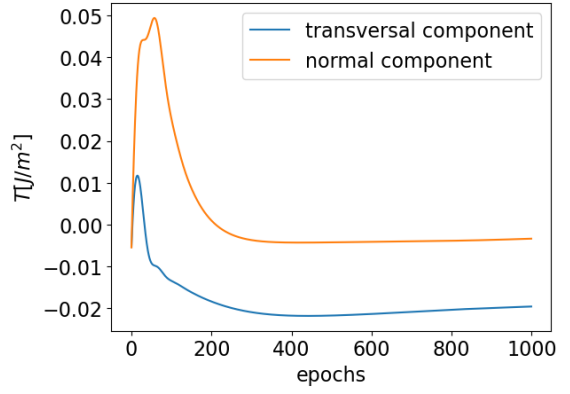
(a) Deformation U_x by 1000 epochs



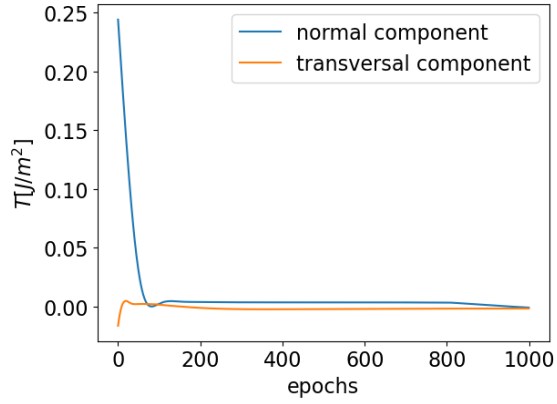
(b) Residual-, T-PSI, boundary, divergence Loss



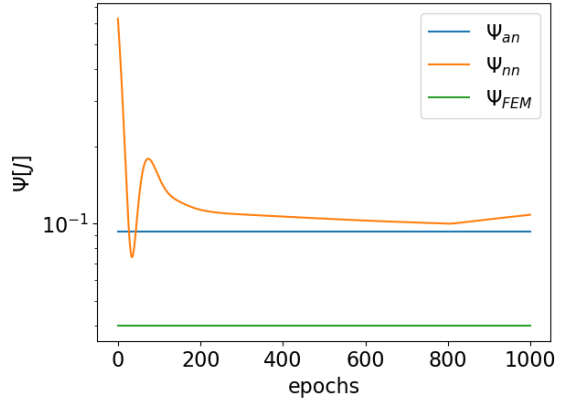
(c) Normal and Transversal Traction on lower face



(d) Normal and Transversal Traction on upper face

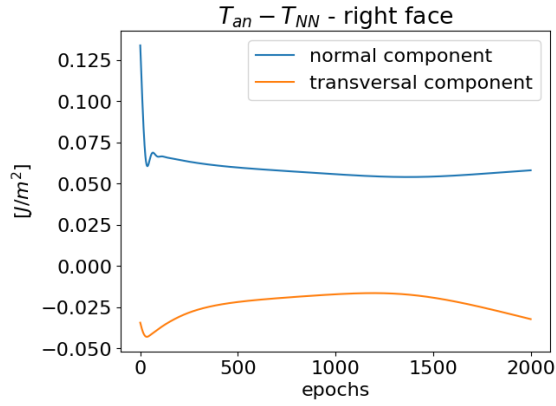


(e) Normal and Transversal Traction on right face

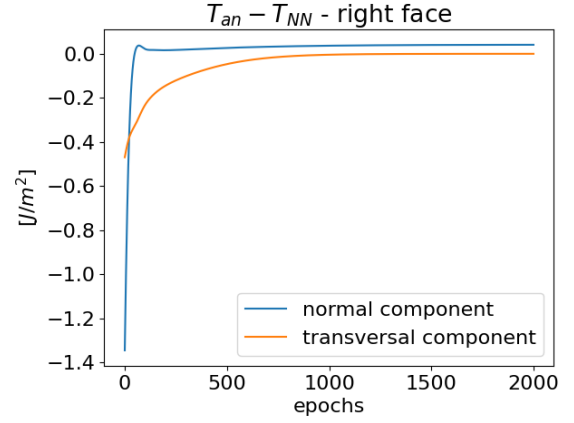


(f) strain energy Ψ

Figure 2.6: Loss, Total energy, Traction with influence of data-driven/physics-informed

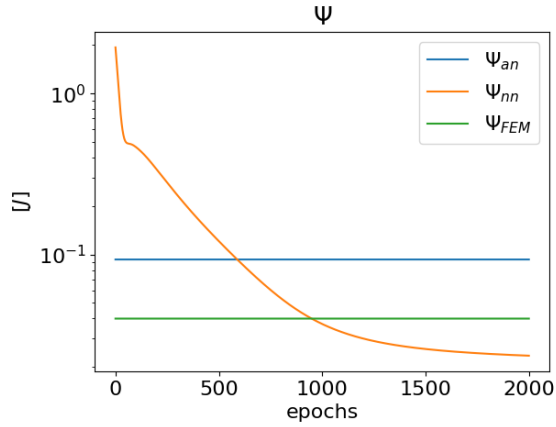


(a) calculation of traction energy using initial force

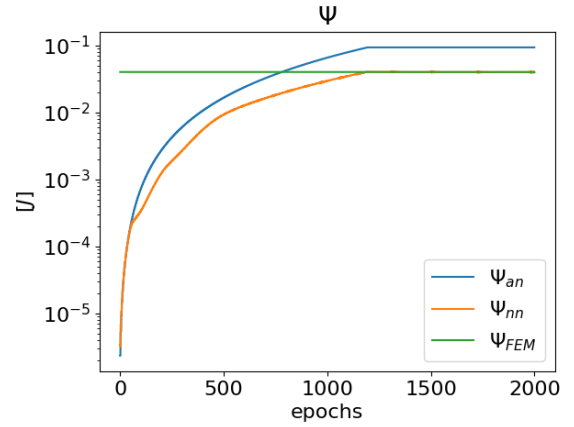


(b) calculation of traction energy using stress

Figure 2.7: comparison of the analytical traction T_{an} and the predicted traction T_{NN} at the right boundary



(a) one step application of $force_{right}$



(b) incremental application of $force_{right}$

Figure 2.8: strain density in dependence of the application of the force boundary condition

2.8 Errors and mistakes

The **first** identified mistake is related to Equation *feh.1*, which concerns the distribution of the global force applied on the right surface to the collocation points defined in `sample_points`. In fact, the distribution of the force should be dependent on the overall length l_y , because, if not, increasing l_y would result in an increased overall applied force with respect to global force, being `forces_right` independent of l_y . Instead, a multiplicative factor of $\frac{\Delta y}{l_y}$ should be considered, in order, first, to have coherent physical units for comparable quantities and, secondly, to consider that, if l_y varies, also the distribution of the force should vary, keeping Δy fixed, in order to have actually

$$\sum_{i=1}^{N_y} F_i = F$$

where N_y is the number of samples considered in y direction and F is the global force applied on the right surface. Otherwise, F could be equivalently defined as q in N m^{-1} , as to have $F_i = q\Delta y$, keeping consistency in terms of the measurement units. Then, the **second** mistake is identified in Equation *feh.2*, supposed to calculate elongation Δl given the applied force. In particular, the whole cross-section should be considered to apply Hooke's law. In fact,

$$\begin{aligned}\sigma &= E \cdot \varepsilon \\ \frac{F}{A} &= E \cdot \frac{\Delta l}{l} \\ \Delta l &= \frac{F \cdot l}{E \cdot A} \\ &= \frac{F \cdot l_x}{E \cdot (l_y \cdot l_z)}\end{aligned}$$

where $l = l_x$.

The third identified problem is that the result of DEM gives coincidence result or keep changing each time doing calculated. This because of DEM is working with weight and bias, which can cause many local minimum or convexity problem. The deformation doesn't perform well.

3 Optimization

In order to analyze the influence of the various hyperparameters and convey a simplified understanding of the behavior of the model, each hyperparameter will be firstly analyzed independently from the others, even if complex interdependencies between them exist. When these manifest a clear physical interpretation, hypothesis will be proposed. Hyperparameter settings in the base configuration are listed in Table 1.

3.1 Integration rule and number of collocation points

3.1.1 Introduction

Pytorch provides a standard implementation of the trapezoidal rule, which is a first order integration rule, which means that can precisely integrate an integrand which follow a linear trend. Considering an higher order integration rule, such as Simpson's rule, can improve the accuracy, which, however, doesn't directly affect the loss value, but, rather, allows for a better translation of the physical quantity used to define the loss, which is an estimation of the former. In fact, when defining the global energy of the body as

$$\Psi = \int_V \psi \, dV \quad (1)$$

Hyperparameter	Base values
Optimizer	Adam
Learning rate	0.001
Number of epochs	2000
Activation function	Tanh
Number of hidden layers	1
Number of neurons per layer	16
Integration rule	Trapezoidal

Table 1: Base configuration for optimization

just a set of finite collocation points are available to evaluate ψ , so the integral in Equation 1 must be numerically estimated with an integration rule. Considering the Taylor expansion theorem for a function of one variable (for simplicity),

$$\psi(x - a) \simeq \psi(a) + \frac{\psi'(a)}{1!}(x - a) + \frac{\psi''(a)}{2!}(x - a)^2 + \dots \quad (2)$$

is immediate to deduce that an higher order integration rule allows to better estimate the analytical integral in Equation 1. Thanks to the integration is possible to express to previous integral as a sum of discrete values that the function f assumes, everyone of which is weighted differently based on the integration rule adopted. As an example, Simpson composite integration rule reads as follows:

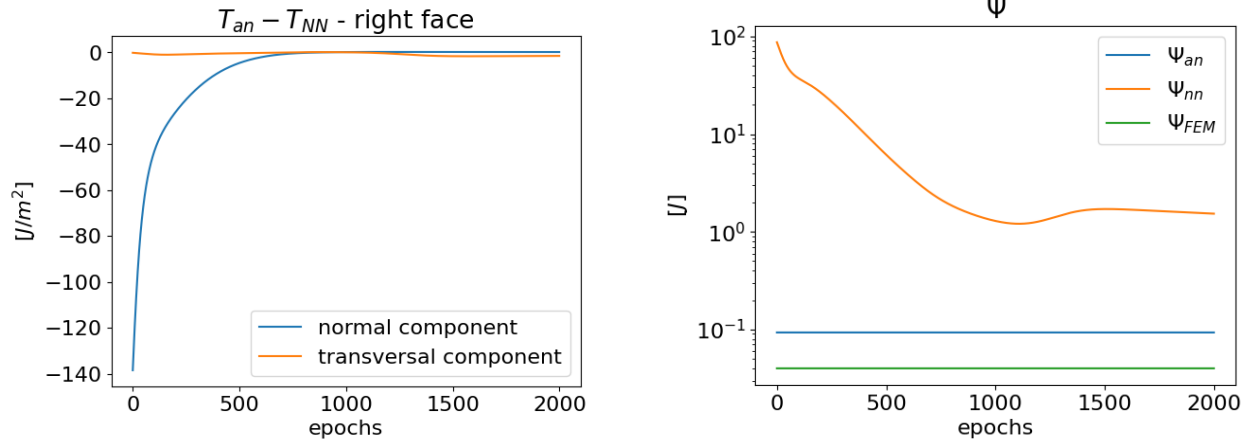
$$\int_a^b f(x) dx \approx \frac{1}{3}h \sum_{i=1}^{n/2} [f(x_{2i-2}) + 4f(x_{2i-1}) + f(x_{2i})] \quad (3)$$

where $h = x_{i+1} - x_i$ assumed constant and n is the number of sampled points, which must be odd. Moreover, for Simpson's rule $E_s = \mathcal{O}(h^4)$ while for the trapezoidal rule $E_t = \mathcal{O}(h^2)$, where E is the error (difference between true value of the integral and result of the quadrature rule). Thus, for example, if the step size h is halved, $E_{t, \frac{h}{2}} = \frac{1}{4}E_t$ while $E_{s, \frac{h}{2}} = \frac{1}{16}E_s$.

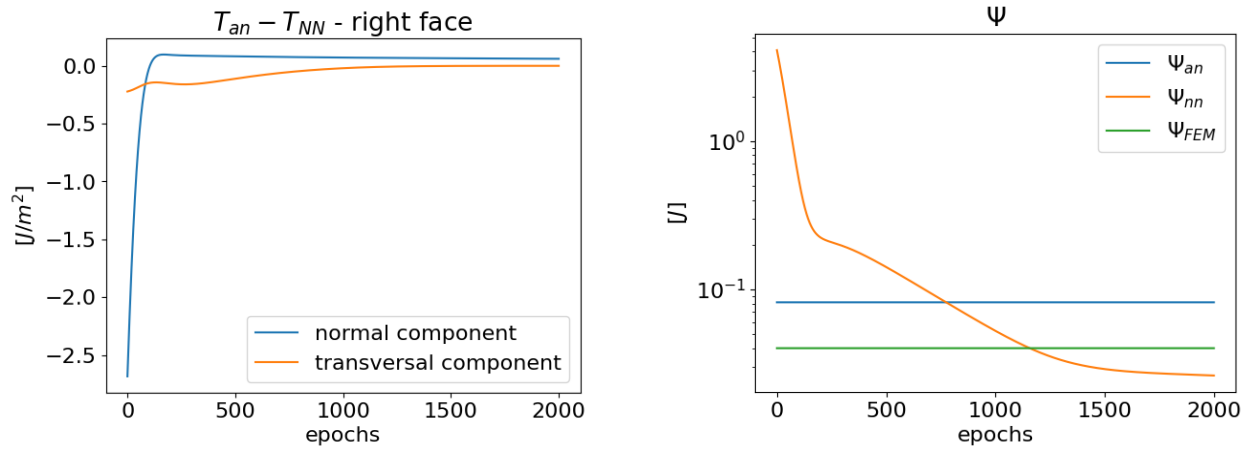
3.1.2 Outcomes

Firstly, the results obtained considering the standard trapezoidal rule and the implemented Simpson's rule will be compared for the physics-informed training, then the model will be first trained for a given number of epochs with a data-driven approach, which will be then changed to the physics-informed approach. Few collocation points will be in general considered, in relation of what explained in ???. Unless otherwise indicated, the weights of all layers will be initialized by means of a scaled normal distribution, in order to limit the influence of a purely random initialization on the results, making them more comparable. The same result, practically, can be achieved training the for a small number of epochs with a data driven approach, in order to pretrain both models, so that the performances of both at the end of the data driven training are more comparable.

Figure 3.1 compares the performance of the two method for a sparse grid and the results are sensibly better for Simpson's rule. Notice that just Ψ_{FEM} is not affected by the quadrature error, being Ψ_{an} calculated feeding `U_target` (line 265 in `DEM_pltTraction`) in the NN. Then, the number of points is increased, so



(a) Trapezoidal rule

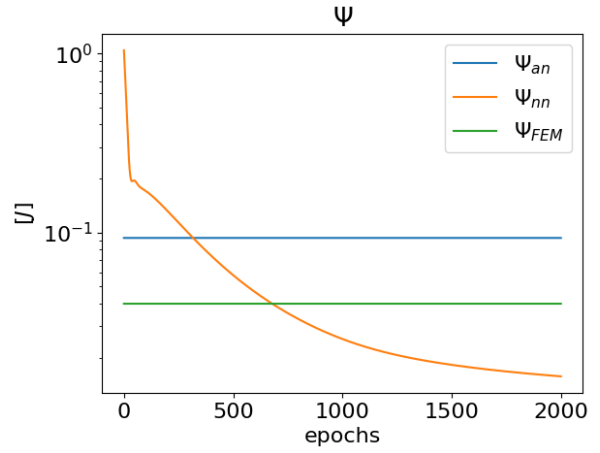
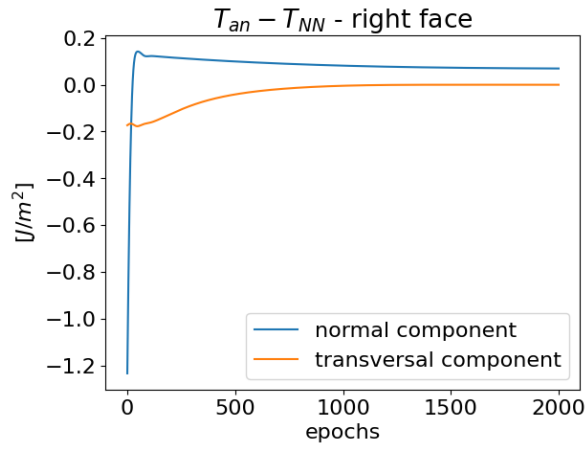


(b) Simpson's rule

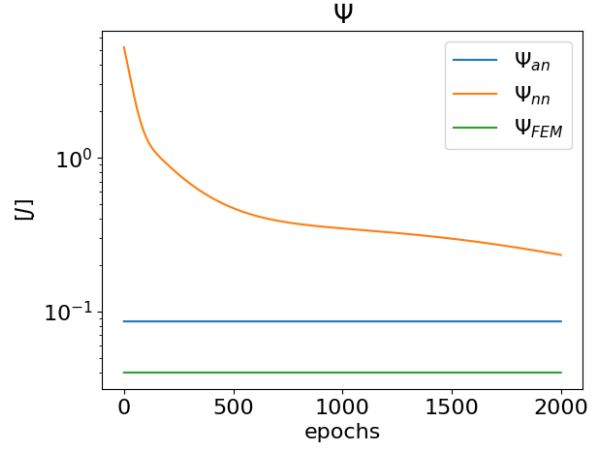
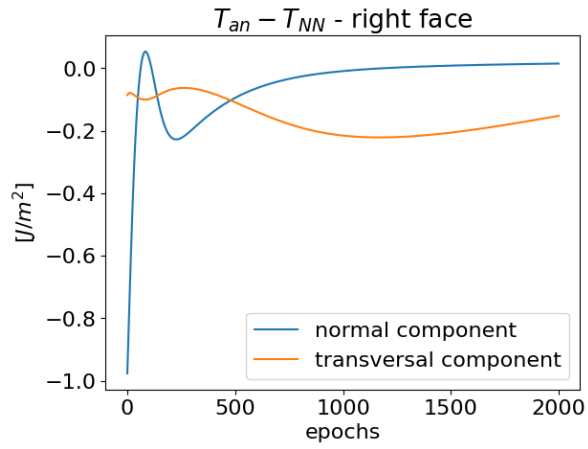
Figure 3.1: `samples_x = samples_y = 9`

the discrepancy between the two methods should decrease, because the step size h decreases, the error for both methods should converge to zero. However, the scaled normal initialization seems to cause convergence problems to the integration with the trapezoidal rule, so, in Figure 3.2, the results obtained using the Simpson's rule with the scaled normal initialization are compared to the ones obtained with the trapezoidal rule and random initialization. The results are reported in Figure 3.3. A sparse mesh is considered in order to appreciate the better outcomes of the Simpson's rule. In particular, the mean displacement of all points lying on a crosssection at x coordinate is much better approximated with the Simpson's rule, while also the energy of the trained model is closer to Ψ_{FEM} .

To summarize, the influence of the integration rule may be relevant when the mesh isn't dense enough as shown in Figure 3.1, but, anyway, the influence of the initialization of weights plays a way more determinant role in the final results.

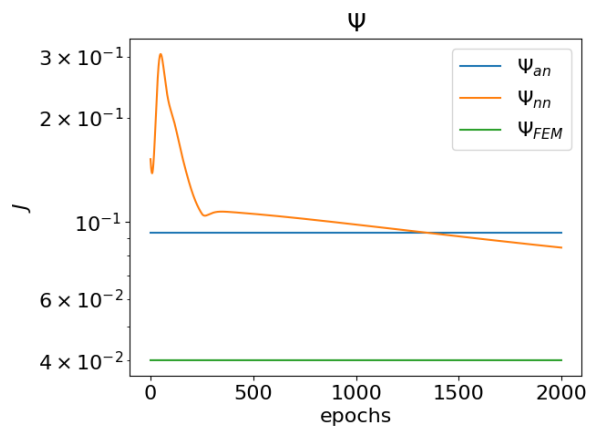
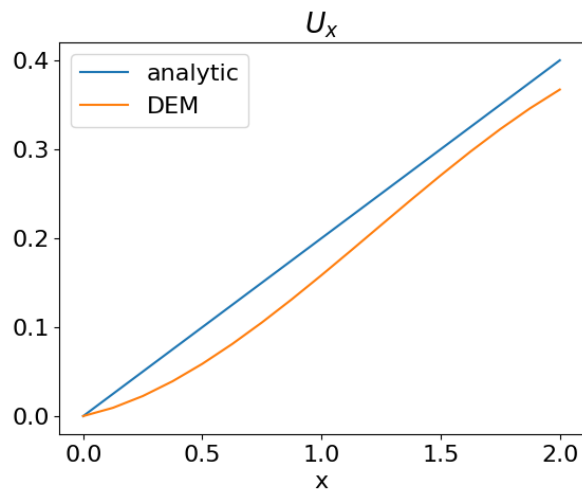


(a) Trapezoidal rule

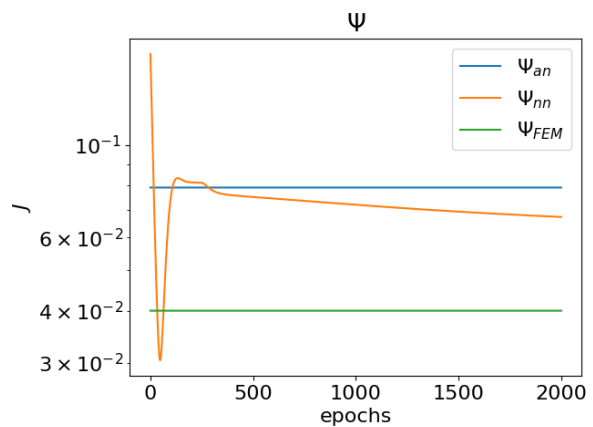
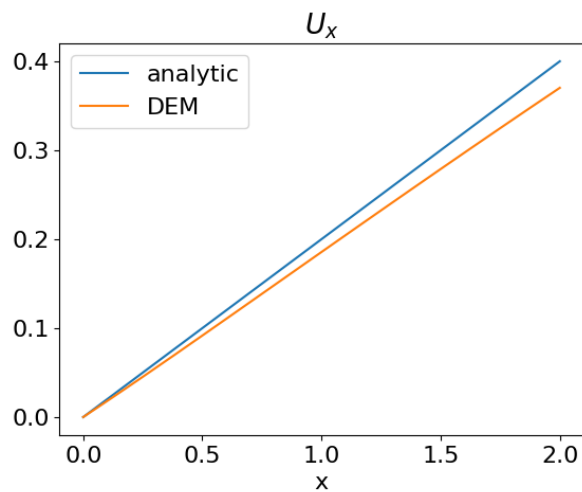


(b) Simpson's rule

Figure 3.2: `samples_x = 17`, `samples_y = 13`



(a) Trapezoidal rule



(b) Simpson's rule

Figure 3.3: 250 epochs with data driven training, `samples_x` = 9, `samples_y` = 7

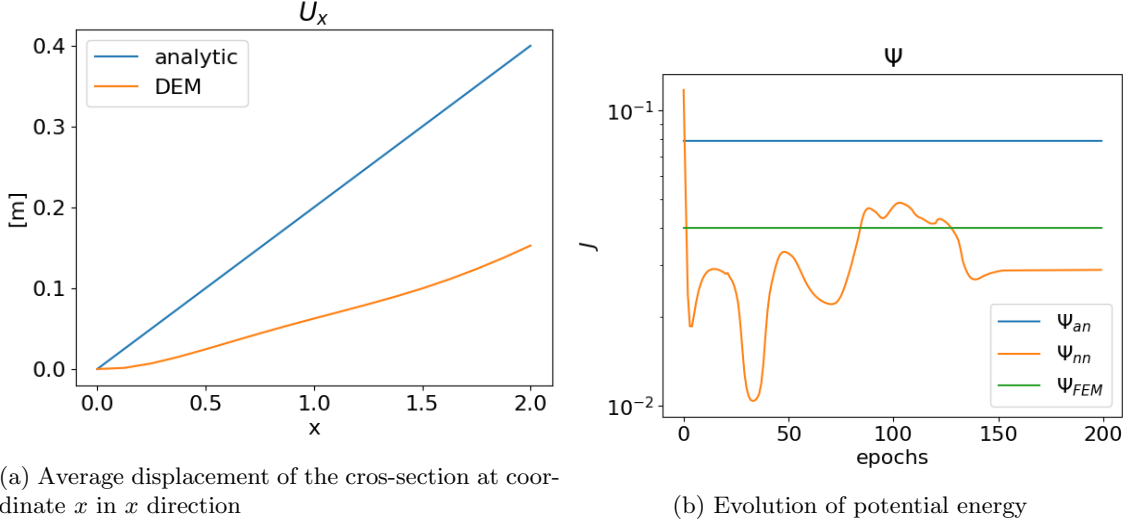


Figure 3.4: LBFGS, 200 epochs, fully physics informed loss

3.2 Optimizer

3.2.1 Introduction

Another hyperparameter choice concerns the optimizer type. The most popular optimizer nowadays is Adam, which tunes the descent rate based on the first and second momentum estimation, which means that the learning rate at the next iteration will be dictated both by the gradients at the current iteration and the accumulated gradients at the past iterations. This balance is controlled by the parameter β_1 , which, if near to 1, gives more weight to the current gradient, and, if near to 0, attributes more importance to the past gradients in determining the step to update the model parameters. Another alternative is the LBFGS optimizer, which is a semi-Newton method, in the sense that approximate the Hessian matrix H , without calculating it directly.

3.2.2 Results

In this section, the standard Adam optimizer will be replaced by LBFGS optimizer. Firstly, the NN is trained fully with a physics-informed approach for 200 epochs. The results are reported in Figure 3.5. Notice that the potential energy converges more rapidly with respect to the results from the Adam optimizer, at a cost of an increased computation time per epoch.

Then, the data-driven loss is considered for the first 20 epochs. The results are reported in Figure 3.6. Notice that, after the initial part of the training where the Ψ varies considerably from epoch to epoch, at the end of the training tends to stabilize, differently from Adam optimizer. Moreover, the average displacement reported in Figure 3.6a is improved when the initial epochs are performed with a data-driven approach, even if the results are not as satisfactory as the Adam optimizer. However, more iterations with the LBFGS are required in order to express a more comprehensive assessment on the performance of the LBFGS optimizer in this specific context.

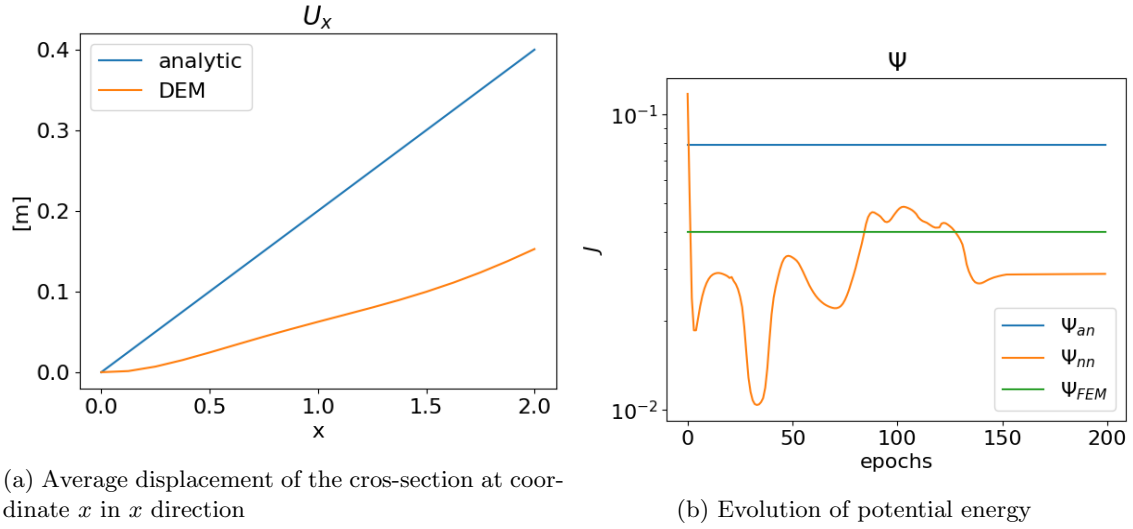


Figure 3.5: LBFGS, 200 epochs, fully physics informed loss

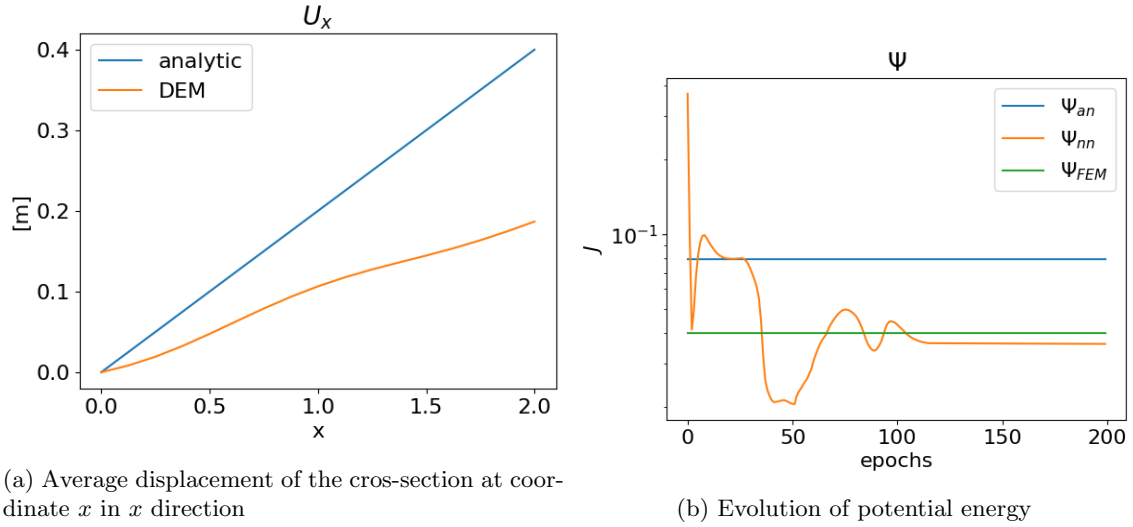


Figure 3.6: LBFGS, 200 epochs, 20 epochs data-driven + 180 epochs physics informed

3.3 Network architecture

3.3.1 Introduction

The objective of a feedforward network is to approximate a continuous function that maps an input to its corresponding output, in accordance with the universal approximation theorem. The neural network is constructed by connecting neurons between layers. The first layer (0th) is the input layer, the last layer (Lth) is the output layer, and the layers in between are the hidden layers. Neurons in each layer are connected to every neuron in the subsequent layer compare Figure 3.7.

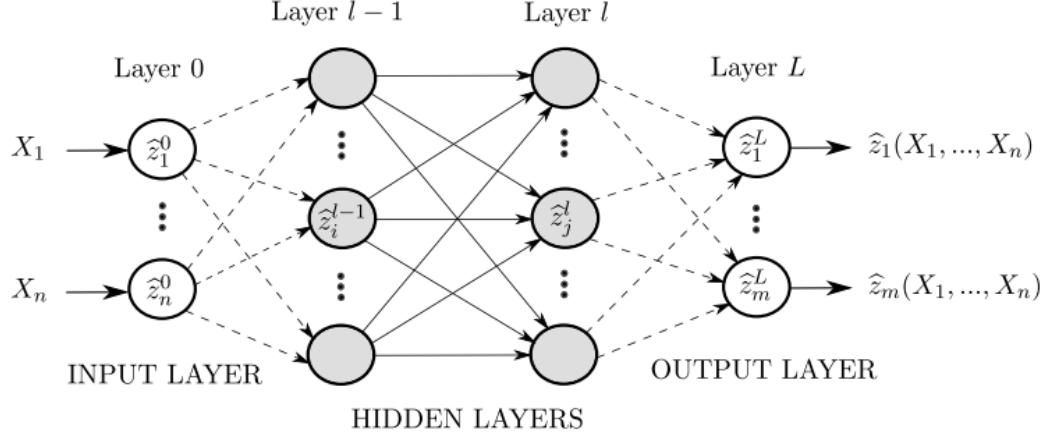


Figure 3.7: general network architecture

3.3.2 Results

In the process of the hyper parameter analysis the experience was that a single hidden layer is enough to cope with this relative simple case, but results in this regard are not yet final. One observation is that additional layers result in more necessary epochs to achieve convergence on the other hand enables higher accuracy. Because of the nonlinear relation between hyper parameter the number of hidden layer is not yet optimized for all cases of materials and sample points.

4 Material modeling

In order to change the material model from isotropic linear elastic material to isotropic compressible Neo-Hookean material, the calculation of the free Helmholtz energy Ψ has to be updated as well as the stress tensor calculation. The equations used to acquire the necessary tensor and scalar fields are as follows:

$$\Psi = \frac{\lambda}{2}(\ln J)^2 - \mu \ln J + \frac{\mu}{2}(I_1 - 2) \quad (4)$$

$$\underline{\underline{\sigma}} = J^{-1}(2C_1(\underline{\underline{F}} - \underline{\underline{F}}^T)) + 2D_1(J - 1)J\underline{\underline{F}}^{-T}\underline{\underline{F}}^T \quad (5)$$

With $C_1 = \frac{\mu}{2}$, $D_1 = \frac{\lambda}{2}$, $J = \det(\underline{\underline{F}})$ and $I_1 = \text{Tr}(\underline{\underline{C}})$

So in order to be able to implement the Neo-Hookean material model the deformation gradient tensor

and Cauchy-Green tensor have to be calculated as well. The free Helmholtz energy and stress Tensor field are then equally used as in the linear elastic case.

In order to cope with the higher complexity of the Neo-Hookean material model in comparison to the linear elastic case, incremental load application is used as well as Simpsons' integration rule. This ensures faster convergence within increments of load applications. An important aspect to mention is that in this hyper-elastic case, there is no analytical solution available that can be used for data-driven learning. But experience showed that a short amount of epochs with data-driven learning with the linear elastic case is a good starting point. The hyperparameter were mostly adapted from previous analysis of the elastic case, but because of the incremental load application and the reduced epochs per load cycle, the learning rate was increased to $lr = 0.01$. This showed good results in combination with a weight decay of $\lambda = 1e - 06$. Other differences are the already mentioned changed integration rule to Simpson-Integration and the number of epochs per load cycle is 1000.

4.1 Results Neo-Hookean material

In order to have a reference point in the assessment of the results from DEM with Neo-Hookean material a FEM analysis with Abaqus was performed. The comparison of the FEM and DEM results in this report will only be qualitative as the results from the DEM analysis are not yet fully optimized. In Figure 4.1 and 4.2 are the deformation fields shown in contour respectively scatter plots, in both cases of U_1 and U_2 is the DEM overestimating the deformation components in comparison to the FEM results. In addition is the effect of transversal contraction much more prevalent in the DEM results.

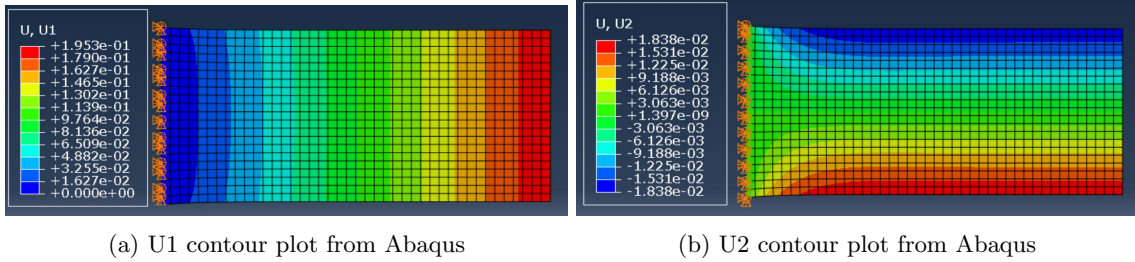


Figure 4.1: FEM deformation fields from 2D Neo-Hookean material analysis

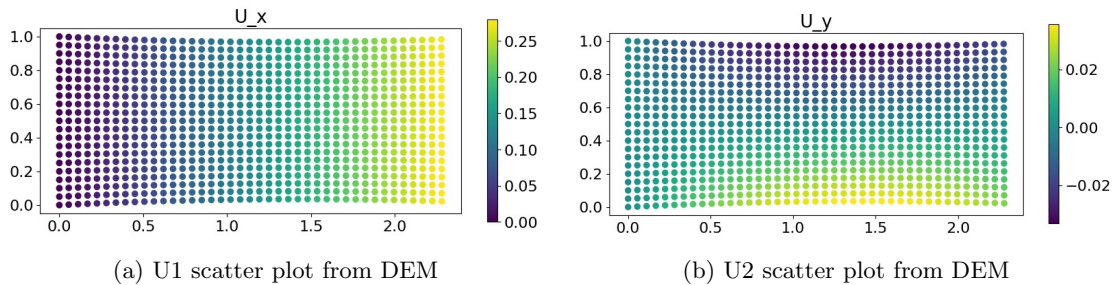


Figure 4.2: DEM deformation fields from 2D Neo-Hookean material analysis

As expected does the changed material properties result in a different free Helmholtz energy as a result from a different deformation field. When comparing Ψ in FEM and DEM analysis in Figure 4.3b, a very good agreement is visible. The evolution of losses in the DEM analysis is depicted in Figure 4.3a and shows pretty quick convergence to an order of $1e - 03$, but fails to further decrease the contributions of losses.

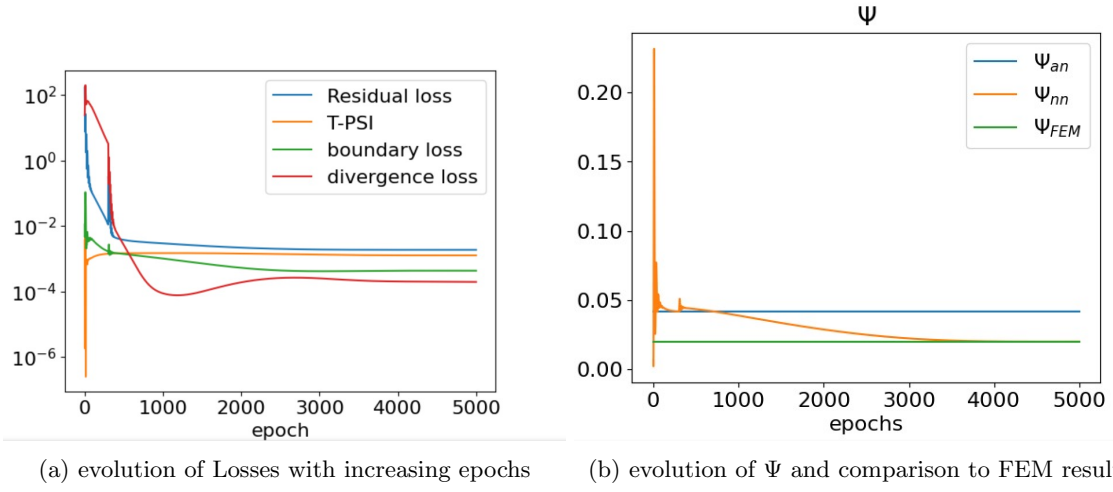


Figure 4.3: DEM analysis performance in regardation of loss and energy minimization

5 Implementation of visual transformer architecture

The effectiveness of transformers in supervised language learning tasks has brought machine learning on the foreground. However, the transformer model, as described in [Vas+23], has not been extensively investigated in the field of scientific computing, above all in a physics-informed context. A reason for that could be its specific construction for discover relationship between objects. In particular, the transformer revolves around the fundamental building block represented by the attention, which is able to discover the underlying relationship between objects in the dataset. An improvement of the attention is the multi-headed attention, which is able to focus on different parts of the input simultaneously, providing an extended set of independent attentions scores, increasing, in this way, the generalizability embedded in the training process.

5.1 Description of the standard visual transformer architecture

The visual transformer represents an adaptation of the classical transformer model, developed in order to analyze pictures and recognize the objects that populate them. As described in [Dos+21], which describes the basic architecture of the visual transformer implementation, the visual transformer firstly process the various images present in the dataset and, for each of them, generate patches, as in Figure 5.2. Then, the flattened patches are passed to the NN, which, in the original implementation, is composed by a linear embedding layer which projects the patches onto a smaller space in order to generate tokens, which will be also positional embedded, in order to retain the order of which the patches appear in the picture. Moreover, to each sequence (which represents a single image, which has been tokenized) an additional special token will be added, in order to extract the global information contained in each picture. Then, the resulting tensor will be passed to the encoding layer, which are composed as described in Figure 5.3. Being the added token a model parameter, it will be trained based on the results of the attention layer, allowing to embed the information and the relationship between all of the patches, which compose a given picture. After the encoder blocks, just the special token will be extracted, and will be fed into a classification MLP, which will output the model's confidence of each class for every sequence, which corresponds to a picture. In this case, the classification MLP will project the tensor from the latent dimension to the classification dimension, which is equal to the number of possible different classes.

The overall data flow is presented in Figure 5.1, where n_x and n_y represent the number of pixel respectively in x and y direction and N_s the flattened patch dimension. For example, for a picture with 3 channels per pixel, $N_s = 3N_{pp}$, where N_{pp} is the number of pixels for a single patch.

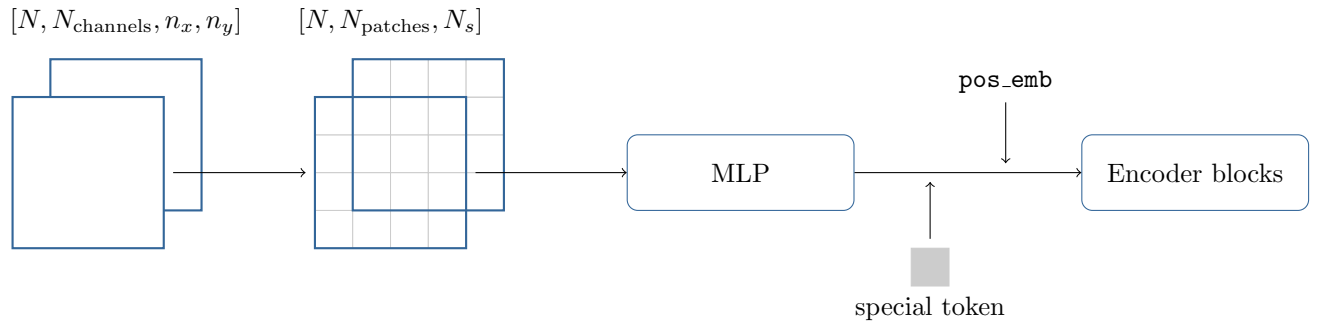


Figure 5.1: Overall data flow in ViT architecture

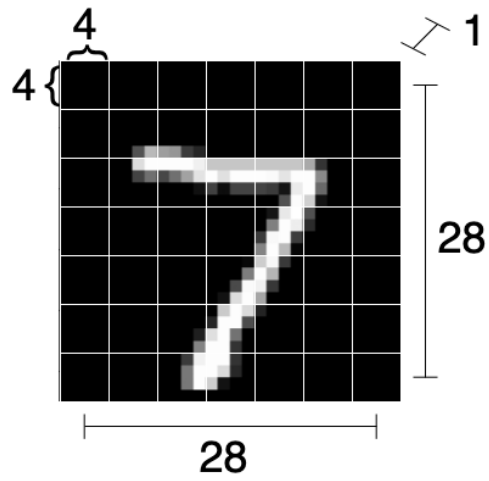


Figure 5.2: Splitting of image into patches (from [Pul22])

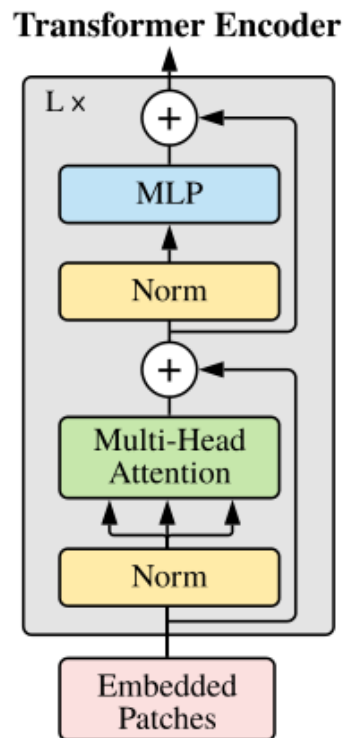


Figure 5.3: Structure of an encoder block of ViT model

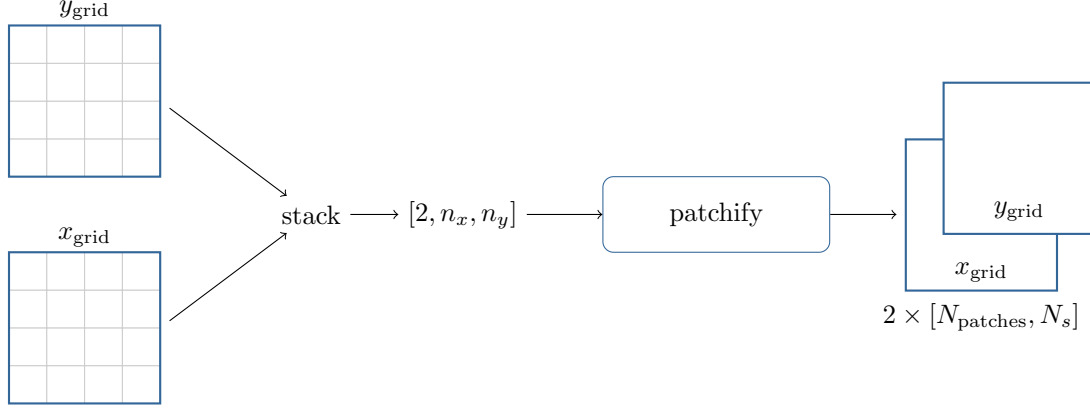


Figure 5.4: Processing of input data

5.2 Adaptation to regression and physics-informed tasks

The ViT architecture has been adapted to handle regression tasks. For that, the softmax activation function, which outputs a probability density function out of a generic distribution of values, useful for classification tasks, has been replaced by a tanh activation function. Before being fed to the ViT, the input is processed as shown in Figure 5.4, where, given N_{patches} , which represent the number of total patches in which to divide the input tensor,

$$N_s = \frac{n_x n_y}{N_{\text{patches}}}$$

which represent the number of points per patch, divided into the x and y coordinate. In this way, superimposing the two vectors that compose the output of **patchify**, a mesh of coordinates is obtained for each patch, as illustrated as last step of Figure 5.4. Notice that for now the **patchify** function has been implemented just for square meshes ($n_x = n_y = n$). This approach is different from the flattening described in subsection 5.1, in order to correctly unpack the patched tensor in the **geteps** function for calculating the displacement gradient with **autograd**. Moreover, given that the global information about a deformed configuration is already embedded in the potential energy Ψ , that can be calculated from the displacement field, the special token would serve the same purpose as the potential energy, and therefore is not considered. Moreover, given that a regression task is considered, all the outputs from the encoder layer will be considered, instead of considering just the special token as in [Dos+21]. Furthermore, positional embedding, which is implemented in the standard architecture of the ViT in order to retain information about the order between the various patches, has not been applied, because the ordinal positional is still retained in the actual entries of the tokenized tensor, as depicted in Figure 5.4.

Note that the attention layers in a physics informed context can serve two different purposes, as described in [LMF23]: discovery of patterns in the spatial domain or in the temporal domain. For this purpose, considering the alternative interpretation of queries, keys and values proposed in [Cao21] where each of the columns $\mathbf{q}_j, \mathbf{k}_j, \mathbf{v}_j$ may be interpreted as the evaluation of the j -th basis function on the grid, attention may be extended as scalar product between two functions $\langle q_j(x), k_s(x) \rangle$, which can be numerically evaluated with a quadrature rule, in order to calculate the attention score between two different basis functions evaluated on the grid as vectors $\mathbf{q}_i$ and $\mathbf{v}_j$, as an example. In this work, however, the standard implementation of the transformer multi-head attention layer is considered.

Hyperparameter	Base values
Optimizer	Adam
Learning rate	0.005
Number of epochs	300
N_{patches}	100
$n_x = n_y = n$	10
Number of encoder blocks	2
Number of attention heads	4
Latent dimension	16

Table 2: Base configuration for ViT

5.3 Results

The hyperparameter settings for the base configuration are reported in Table 2. Moreover, the Dirichlet boundary conditions have been enforced through a transformation as a last step of the forward function.

Firstly, the model in the base configuration is considered. The global metrics using for quantitative assessment of the performances are reported in Figure 5.6, while the field results are listed in Figure 5.7. Notice that the qualitative distribution of the displacement at the upper and lower boundaries of the domain resembles the not-linear trend evidenced also from the FEM results reported in Figure 5.5, due to the full planar stress state deriving from the Dirichlet boundary conditions. Moreover, Figure 5.6a reveals that the two equivalent loss terms related respectively to $T - \Psi$ and $\nabla \cdot \sigma$ follow the same trend, even if just $T - \Psi$ is being optimized. In this way, given that for an equilibrium configuration the two statements are equivalent, the robustness of the training process is assessed. Also notice that in the base case, $N_s = 1$, so one patch is made up only by one point.

Then, N_{patches} is decreased to 25 and the epochs to 200. When $n^2 > N_{\text{patches}}$, which means that $N_s > 1$, every token (which corresponds to a patch), which now is not just a point, but a set of points, contributes independently to the reconstruction of the displacement field, and therefore the initial displacement field looks as composed by $\frac{N_{\text{patches}}}{n^2}$ grids, as noticeable in Figure 5.8a. However, even if the trained displacement field as shown in Figure 5.8b results, basically, from the superimposition of more grids, and, therefore, more irregular, the qualitative non-linear shape is still outlined. The energy trend along training is reported in Figure 5.8c, which shows good convergence to Ψ_{FEM} , even with relatively few iterations.

One potential disadvantage of this implementation is the increased computational time, due to the presence of Q, K and V as trainable parameters, along with an MLP, for each encoder block. However, this shortcoming is balanced by the fact that convergence is reached faster than classical MLPs, as can be seen comparing the evolution of energy along the training process. Another identified problem that needs to be further addressed is the segmentation of the grid resulting from the patching process, which produces several independent grids, which directly influence the outcomes of the training process, which reflect the segmentation of the initial grid.

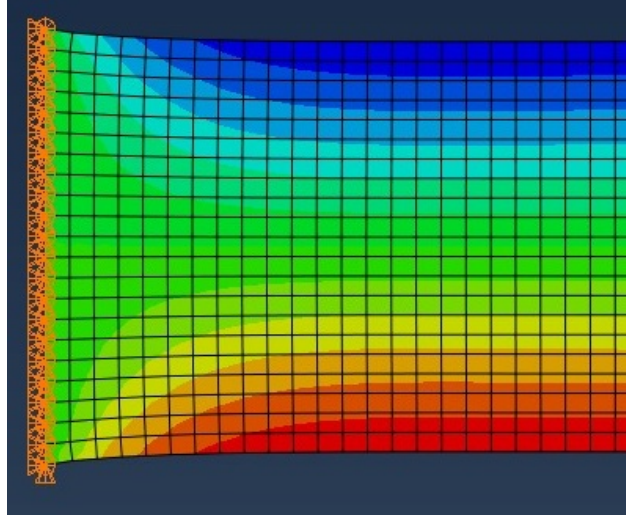
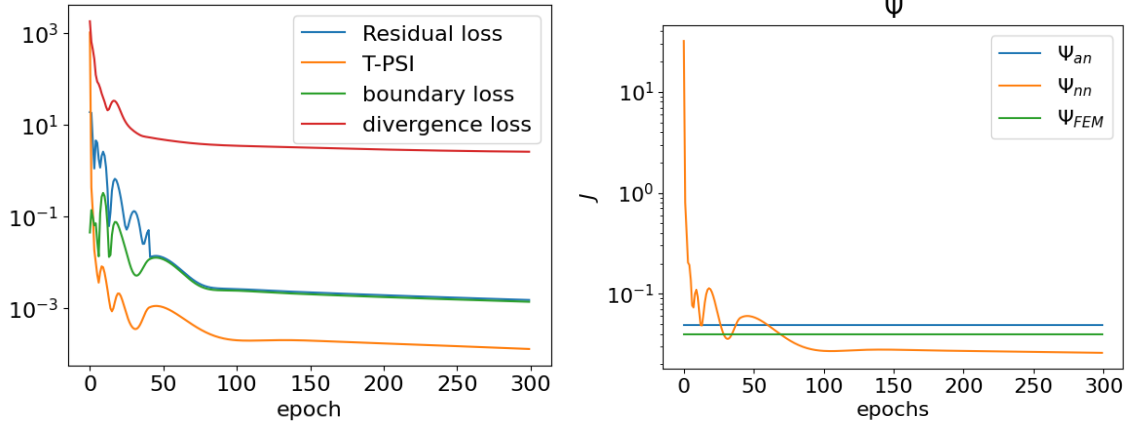
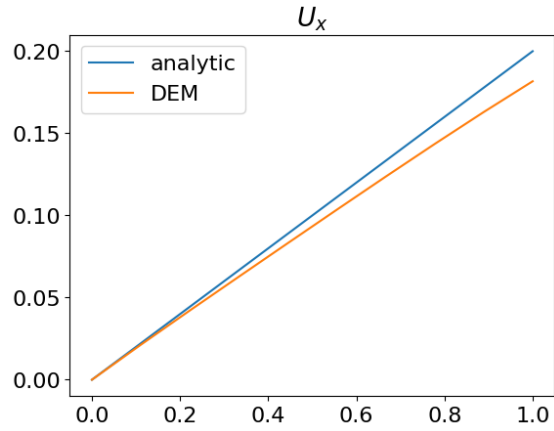


Figure 5.5: FEM results for the given boundary conditions



(a) Loss curves

(b) Evolution of potential energy



(c) Average displacement in x direction

Figure 5.6: ViT in base configuration

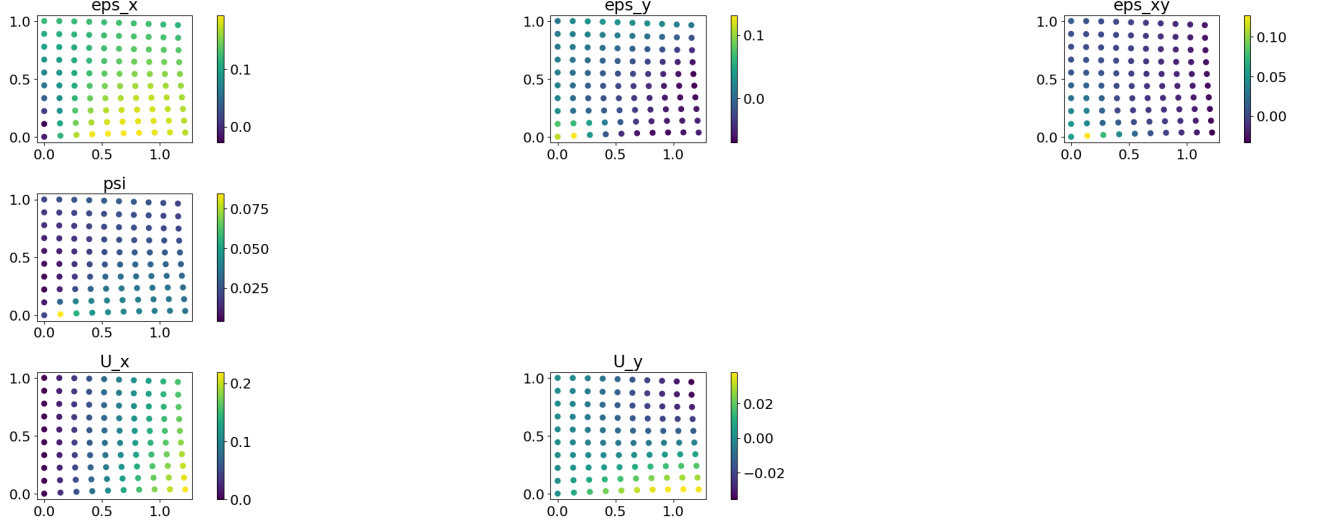


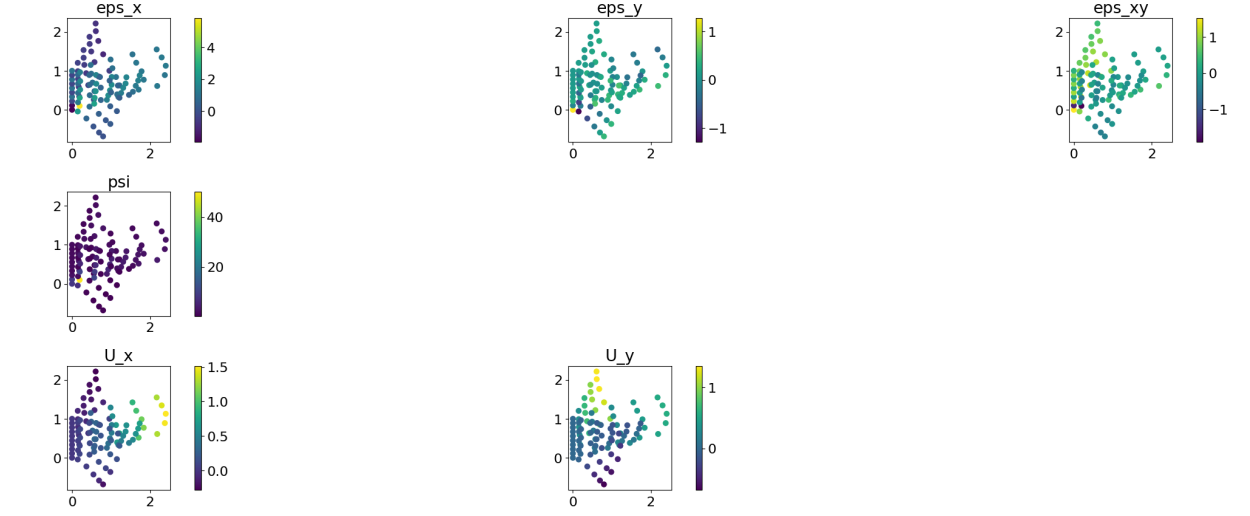
Figure 5.7: Fields for trained ViT in base configuration

5.4 Alternative architecture as graph transformer

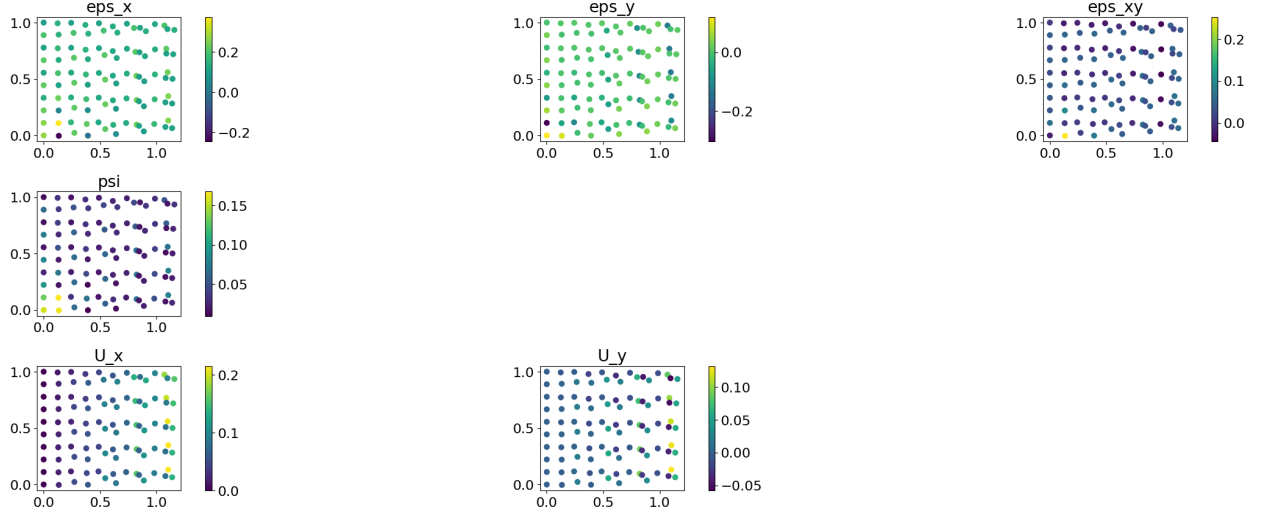
The effectiveness of transformers in discovering relationship between objects, leveraging the attention mechanism, and in particular the multi-headed attention, are key factors for increasing the generalizability of the predictions. The employed adaptation of the visual transformer on the other side was found to be counter intuitive in the aspect of the encoding of the raw data. The representation and adaptation of positional data to cohere to the structure of an image was found to be forceful, needing several data manipulation before being fed to the model. Moreover, as discussed in 5.3, the segmentation of the input grid into patches would require, in traditional terms, the introduction of the positional embedding of each patch.

A more intuitive approach would be to encode the input data in a graph data structure, where the nature of positional data is preserved, and the mutual influence between discrete points in the graph neural network (GNN) can be considered at ease [FZ24]. GNNs and, in particular, Physics-informed GNNs [GZW22] possess the capability to consider the relationships between objects, mirroring the observation that material points in close proximity generally exhibit similar physical properties, opening the possibility to handle materials without uniform properties. The aforementioned architecture has been theoretically explored in this report by now.

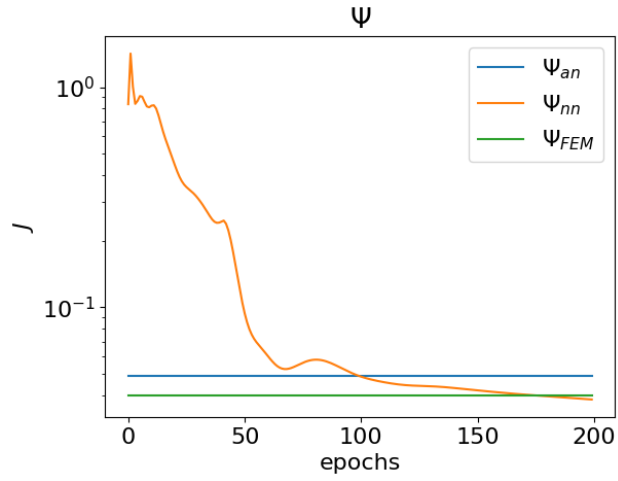
The Energy-informed Graph Transformer model proposed in [FZ24] leverages the multi-head attention mechanism to consider the relationship between objects of graph-structured data, following an Encode-Process-Decode architecture as shown in Figure 5.9. The input grid is modeled as a Graph composed of nodes and edges, where each nodes encode the spatial coordinates, and edges encode the relative coordinates of two adjacent nodes and their distance. The feature space is then augmented through a MLP Encoder and the augmented data is passed to a multi-head attention residual network. Important here is the concept of aggregation in each step of the message passing block. Here the target object’s features are iteratively considered together with the ones of adjacent objects, thus augmenting the size of the considered neighbourhood at every iteration. Finally the decoder utilizes an MLP to transform the aggregated node features (of primary interest) into one or more output features.



(a) Fields before training for $n^2 > N_{\text{patches}}$



(b) Fields after training for $n^2 > N_{\text{patches}}$



(c) Evolution of energy

Figure 5.8: ViT with $n^2 > N_{\text{patches}}$

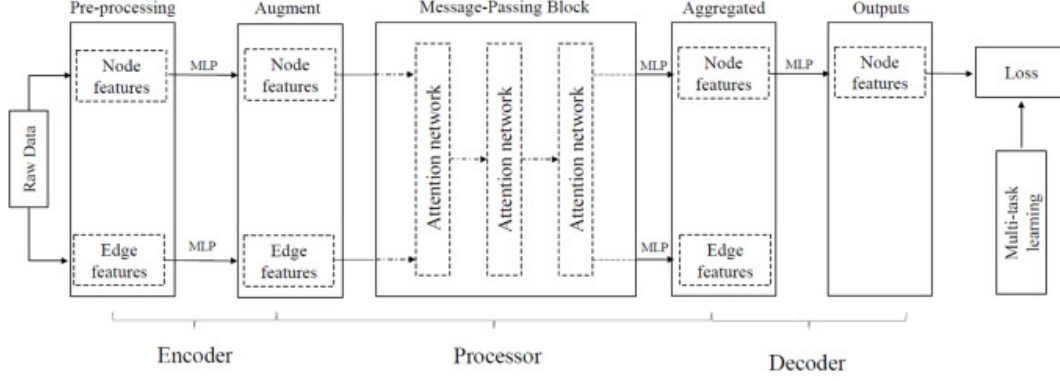


Figure 5.9: Representation of the Encode-Process-Decode architecture for graph neural networks.

References

- [Cao21] Shuhao Cao. *Choose a Transformer: Fourier or Galerkin*. 2021. arXiv: 2105.14995 [cs.LG]. URL: <https://arxiv.org/abs/2105.14995>.
- [Dos+21] Alexey Dosovitskiy et al. *An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale*. 2021. arXiv: 2010.11929 [cs.CV]. URL: <https://arxiv.org/abs/2010.11929>.
- [FZ24] Bo Feng and Xiaoping Zhou. “Energy-informed graph transformer model for solid mechanical analyses”. In: *Communications in Nonlinear Science and Numerical Simulation* 137 (2024), p. 108103. ISSN: 1007-5704. DOI: <https://doi.org/10.1016/j.cnsns.2024.108103>. URL: <https://www.sciencedirect.com/science/article/pii/S1007570424002880>.
- [GZW22] Han Gao, Matthew J Zahr, and Jian-Xun Wang. “Physics-informed graph neural Galerkin networks: A unified framework for solving PDE-governed forward and inverse problems”. In: *Computer Methods in Applied Mechanics and Engineering* 390 (2022), p. 114502.
- [LMF23] Zijie Li, Kazem Meidani, and Amir Barati Farimani. *Transformer for Partial Differential Equations’ Operator Learning*. 2023. arXiv: 2205.13671 [cs.LG]. URL: <https://arxiv.org/abs/2205.13671>.
- [Pul22] Brian Pulfer. *Vision Transformers from Scratch (PyTorch): A step-by-step guide*. <https://medium.com/@brianpulfer/vision-transformers-from-scratch-pytorch-a-step-by-step-guide-96c3313c2e0c>. 2022.
- [Vas+23] Ashish Vaswani et al. *Attention Is All You Need*. 2023. arXiv: 1706.03762 [cs.CL]. URL: <https://arxiv.org/abs/1706.03762>.